

UTN San Nicolás - Tecnicatura En Programación A Distancia

Comisión 4 - Pichulman Miguel; Rodriguez Joaquin

TPI Integrador Base de datos II - TPI-G013

27/5/26

índice

Modelado de Datos NoSQL: Sistema de Autenticación y Gestión de Sesiones.....	3
Definición del problema de negocio	3
Especificación del modelo de datos (esquema JSON)	4
Datos de ejemplo	7
Justificación técnica de las decisiones de modelado.....	9
Uso de embedding (desnormalización) en la colección usuario	9
Uso de linking en la relación usuario–credencial (1:1).....	9
Uso de linking en la relación usuario–sesiones (1:N).....	9
Implementación de baja lógica.....	10
Configuración del entorno de trabajo.	11
Conexion para Compass Docente e tutores	15

Modelado de Datos NoSQL: Sistema de Autenticación y Gestión de Sesiones

Definición del problema de negocio

En el desarrollo de aplicaciones web modernas, la gestión de identidades, el control de accesos y el seguimiento de sesiones activas constituyen componentes fundamentales para garantizar la seguridad del sistema y una experiencia de usuario eficiente.

El presente trabajo aborda el diseño de un modelo de datos NoSQL orientado a un sistema de autenticación y gestión de sesiones, implementado en MongoDB Atlas. El problema de negocio se centra en construir una estructura de persistencia que permita:

Registrar usuarios con sus datos de perfil.

Almacenar credenciales de autenticación de manera segura.

Gestionar sesiones activas y su ciclo de vida.

Mantener la integridad lógica de los datos mediante eliminación no destructiva (baja lógica).

Este enfoque busca asegurar escalabilidad, rendimiento y seguridad en operaciones críticas del sistema, especialmente aquellas relacionadas con el acceso de usuarios.

Especificación del modelo de datos (esquema JSON)

El modelo propuesto está compuesto por tres colecciones principales, diseñadas bajo un enfoque documental propio de MongoDB:

Colección: usuario

Esta colección representa la entidad central del sistema y almacena la información principal del usuario.

Incluye datos personales y de perfil necesarios para la identificación dentro de la aplicación.

```
db.createCollection("usuario", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["email", "perfil", "fecha_registro",
"activo"],
      properties: {
        email: {
          bsonType: "string",
          pattern: "^.+@.+\\..+$", // validación básica de
email
          description: "Debe ser un email válido"
        },
        perfil: {
          bsonType: "object",
          required: ["nombre", "apellido", "telefono"],
          properties: {
            nombre: { bsonType: "string" },
            apellido: { bsonType: "string" },
            telefono: { bsonType: "string" }
          }
        },
        fecha_registro: { bsonType: "date" },
        activo: { bsonType: "bool" },
        fecha_baja: { bsonType: ["date", "null"] }
      }
    }
  }
})
```

Colección: credencial

Contiene la información relacionada con la autenticación del usuario.

Cada documento se asocia de forma unívoca a un usuario mediante un identificador (usuario_id), almacenando datos sensibles como el hash de la contraseña.

```
db.createCollection("credencial", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["usuario_id", "password_hash"],
      properties: {
        usuario_id: {
          bsonType: "objectId",
          description: "Debe referenciar al _id de usuario"
        },
        password_hash: {
          bsonType: "string",
          description: "Hash seguro de la contraseña"
        }
      }
    }
  }
})
```

Colección: sesiones

Registra cada inicio de sesión exitoso del usuario, permitiendo el seguimiento de la actividad y el control del ciclo de vida de cada sesión.

Incluye información como fecha de inicio, expiración y estado de validez.

```
db.createCollection("sesiones", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["usuario_id", "fecha_creacion",
"fecha_expiracion", "estado"],
      properties: {
        usuario_id: {
          bsonType: "objectId",
          description: "Debe referenciar al _id de usuario"
        },
        fecha_creacion: { bsonType: "date" },
        fecha_expiracion: { bsonType: "date" },
        estado: {
          enum: ["activa", "revocada", "expirada"],
          description: "Estado de la sesión"
        }
      }
    }
  }
})
```

Datos de ejemplo

```
>_ mongosh: tpidb.2yeanu... LoginTPI sesiones credencial +
>_MONGOSH
> db.usuario.insertOne({
  _id: ObjectId("60d5ec49f1b2c8000000000001"),
  email: "cliente@ejemplo.com",
  perfil: {
    nombre: "Juan",
    apellido: "Pérez",
    telefono: "+54 9 261 123456"
  },
  fecha_registro: ISODate("2026-05-27T10:00:00Z"),
  activo: true,
  fecha_baja: null
})

db.credencial.insertOne({
  _id: ObjectId("60d5ec55f1b2c8000000000002"),
  usuario_id: ObjectId("60d5ec49f1b2c8000000000001"),
  password_hash: "$2b$10$w...hash_seguro..."
})

db.sesiones.insertOne({
  _id: ObjectId("60d5ec60f1b2c8000000000003"),
  usuario_id: ObjectId("60d5ec49f1b2c8000000000001"),
  fecha_creacion: ISODate("2026-05-27T17:00:00Z"),
  fecha_expiracion: ISODate("2026-05-28T17:00:00Z"),
  estado: "activa"
})
< {
```

```
acknowledged: true,
insertedId: ObjectId('60d5ec60f1b2c8000000000003')
}
Atlas atlas-o2sp4m-shard-0 [primary] LoginTPI>
```

credencial +

tpidb.2yeaunuh.mongodb.net > LoginTPI > credencial

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId('60d5ec55f1b2c80000000002'),
  "usuario_id": ObjectId('60d5ec49f1b2c80000000001'),
  "password_hash": "$2b$10$w...hash_seguro..."
}
```

sesiones +

tpidb.2yeaunuh.mongodb.net > LoginTPI > sesiones

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId('60d5ec60f1b2c80000000003'),
  "usuario_id": ObjectId('60d5ec49f1b2c80000000001'),
  "fecha_creacion": "2026-05-27T17:00:00.000+00:00",
  "fecha_expiration": "2026-05-28T17:00:00.000+00:00",
  "estado": "activa"
}
```

usuario +

tpidb.2yeaunuh.mongodb.net > LoginTPI > usuario

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId('60d5ec49f1b2c80000000001'),
  "email": "cliente@ejemplo.com",
  "perfil": Object,
  "fecha_registro": "2026-05-27T10:00:00.000+00:00",
  "activo": true,
  "fecha_baja": null
}
```


Justificación técnica de las decisiones de modelado

A diferencia del modelo relacional tradicional, MongoDB se basa en el diseño orientado a patrones de acceso. En este contexto, las decisiones de modelado se fundamentan en criterios de rendimiento, seguridad y escalabilidad.

Uso de embedding (desnormalización) en la colección usuario

Se aplicó embedding al incluir el subdocumento perfil (nombre, apellido, teléfono) dentro del documento de la colección usuario.

Esta decisión se fundamenta en los siguientes aspectos:

- **Optimización de rendimiento:** la información del perfil es consultada frecuentemente en operaciones de lectura, como la visualización del nombre de usuario en la interfaz o formularios de interacción.
- **Reducción de joins:** al evitar operaciones de tipo **\$lookup**, se minimiza la latencia y se mejora el tiempo de respuesta del sistema.
- **Modelo de relación 1:1:** los datos de perfil dependen directamente del usuario y comparten su ciclo de vida.

En consecuencia, la desnormalización mediante embedding permite consultas más eficientes en escenarios de alta concurrencia.

Uso de linking en la relación usuario–credencial (1:1)

Aunque la relación entre usuario y credencial es de tipo uno a uno, se optó por una estrategia de linking mediante la referencia `usuario_id`.

Esta decisión se justifica por motivos de seguridad y optimización del modelo:

- **Protección de datos sensibles:** el campo `password_hash` se aísla en una colección independiente, reduciendo el riesgo de exposición en consultas generales.
- **Principio de menor exposición:** evita que información crítica sea devuelta en operaciones que no requieren autenticación.
- **Optimización del working set:** la colección usuario permanece liviana, favoreciendo el rendimiento en operaciones frecuentes.

Uso de linking en la relación usuario–sesiones (1:N)

La relación entre usuario y sesiones se modela mediante linking, donde cada documento de sesión contiene el campo `usuario_id`.

Esta decisión responde a criterios estructurales y de escalabilidad:

- **Evitar arrays no acotados:** el uso de embedding generaría un crecimiento ilimitado del documento usuario, afectando el rendimiento.
- **Límite de tamaño de documento:** MongoDB impone un máximo de 16 MB por documento.
- **Alta frecuencia de escritura:** las sesiones se generan constantemente, lo que haría ineficiente su almacenamiento embebido.
- **Mejor escalabilidad:** permite consultas independientes sobre sesiones sin afectar el documento principal.

Implementación de baja lógica

Se implementa un mecanismo de eliminación no destructiva mediante los campos:

- activo (booleano)
- fecha_baja (timestamp)

Cuando un usuario solicita la eliminación de su cuenta, el sistema actualiza el documento estableciendo:

- activo: false
- fecha_baja: [fecha actual]

Este enfoque permite:

- Mantener la integridad histórica de los datos.
- Evitar pérdida de información crítica para auditoría.
- Garantizar consistencia referencial entre colecciones.
- Cumplir con buenas prácticas de sistemas auditables.

Configuración del entorno de trabajo.

1- En MongoDB Cloud creamos una nueva Organization

?

M

Organizations

Create New Organization

[← Back to Organizations](#)

Create Organization

1

2

Name and ServiceMembers and Security

Name Your Organization

TPI - Base de Datos2

Select Cloud Service

Features	☒ MongoDB Atlas	☐ Cloud Manager
Automated database configuration	✓	✓
Continuous backup and point-in-time recovery	✓	✓
Queryable backup snapshots	✓	✓
Fine grained database monitoring & customizable alerts	✓	✓
Real-time performance panel	✓	✓
Data explorer	✓	✓
Automated cluster lifecycle management	✓	

2-Agregamos miembros y establecemos permisos

Add Members and Set Permissions

Invite new or existing users via email address...

Give your members access permissions below. [See full list of organization roles.](#)

miguel.a.pichulman@gmail.com (you)

Organization Owner ✕

Remove

jo.a.cobell23@gmail.com

Organization Member ✕

Organization Owner ✕

Remove

Add Security Contact Information

The designated Atlas security contact will receive security-related notifications, including notifications from the MongoDB Security Team. Note that this contact point is for notifications only and is not authorized for security decisions or approvals. [Learn More](#)

Email address

Optional

Require IP Access List for the Atlas Administration API



This will force all Atlas Administration API operations for your organization to originate from an IP Address added to your Access List by the API Key. [Docs](#)

3-Creamos un Proyecto y establecemos permisos



No Projects Yet

To get started, create a project. Projects act as secure containers for clusters and resources in Atlas.

Create new project

Create a Project



Add Members and Set Permissions

Invite new or existing users via email address...

Give your members access permissions below. [See full list of project roles.](#)

miguel.a.pichulman@gmail.com
m (you)

Project Owner ✕

Remove

jo.a.cobel123@gmail.com

Project Read Only ✕



Remove

Back

Cancel

Create Project

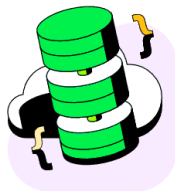
5-Creamos un cluster

ORGANIZATION
TPI - Base de Datos2

PROJECT
TP Integrador

?

TP Integrador Overview



Create a cluster

Choose your cloud provider, region, and specs.

+ Create

Toolbar

Featured Resources

GENERAL

[Get Started with Atlas](#)

[Reference MongoDB Documentation](#)

[Learn MongoDB Skills](#)

[Atlas Learning Hub](#)

New On Atlas

Learn about the latest feature enhancements on Atlas.

Support

Support Plan: Basic Plan

CHAT WITH SUPPORT

VIEW SUPPORT OPTIONS

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ M10

\$0.12/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

☐ Flex

From \$0.011/hour
Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

☒ Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

✓ Free forever!

Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name

You cannot change the name once the cluster is

Quick setup

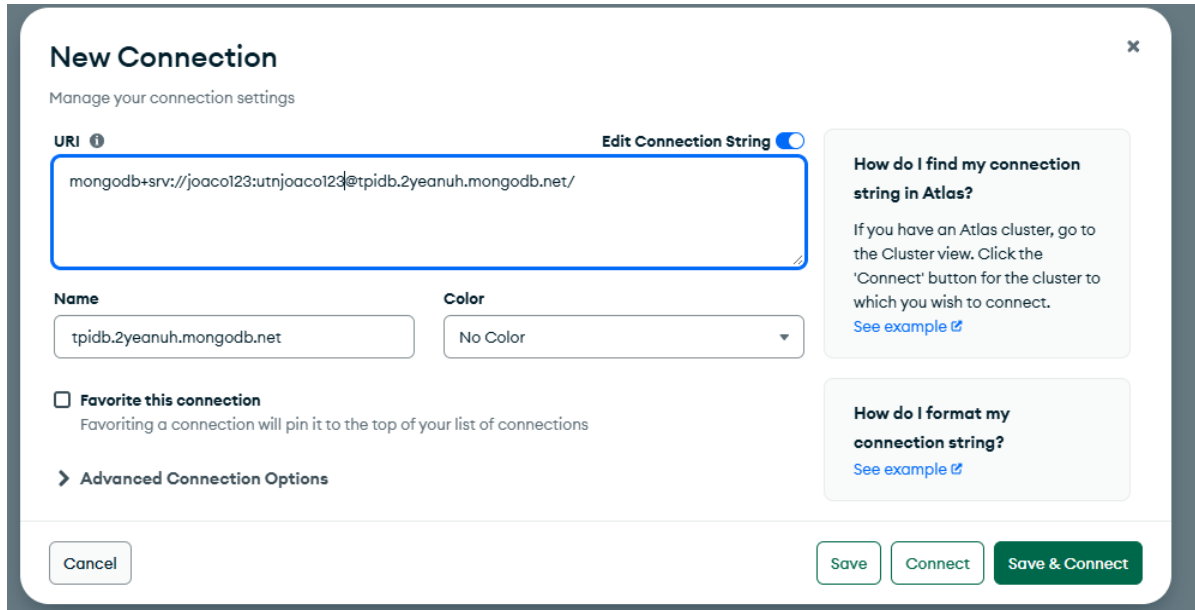
☒ Automate security setup

I'll do this later

Go to Advanced Configuration

Create Deployment

6-Conectamos el cluster



The screenshot shows the 'New Connection' dialog box in MongoDB Compass. The dialog has a title bar with a close button (X). Below the title, it says 'Manage your connection settings'. The main section is titled 'URI' and contains a text input field with the connection string 'mongodb+srv://joaco123:utnjoaco123@tpidb.2yeauh.mongodb.net/'. To the right of the input field is a toggle switch labeled 'Edit Connection String'. Below the URI field, there are two sections: 'Name' with a text input field containing 'tpidb.2yeauh.mongodb.net', and 'Color' with a dropdown menu showing 'No Color'. Below these is a checkbox labeled 'Favorite this connection' with a subtext 'Favoriting a connection will pin it to the top of your list of connections'. At the bottom left, there is a link '> Advanced Connection Options'. At the bottom right, there are three buttons: 'Cancel', 'Save', and 'Connect'. A 'Save & Connect' button is also present. On the right side of the dialog, there are two informational boxes. The first is titled 'How do I find my connection string in Atlas?' and contains text about finding the cluster in the Atlas interface, with a link 'See example'. The second is titled 'How do I format my connection string?' and contains a link 'See example'.

New Connection

Manage your connection settings

URI Edit Connection String

mongodb+srv://joaco123:utnjoaco123@tpidb.2yeauh.mongodb.net/

Name

tpidb.2yeauh.mongodb.net

Color

No Color

☐ **Favorite this connection**

Favoriting a connection will pin it to the top of your list of connections

> **Advanced Connection Options**

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

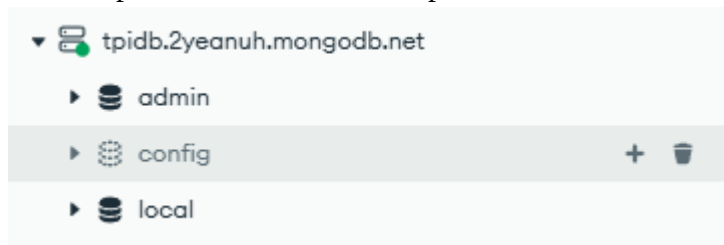
[See example](#)

How do I format my connection string?

[See example](#)

Cancel Save Connect Save & Connect

7- Nos aparecerá el cluster, listo para desarrollar la base de datos en Compass.



Conexion para Compass Docente e tutores

mongodb+srv://profes4:utncom4@tpidb.2yeauh.mongodb.net/